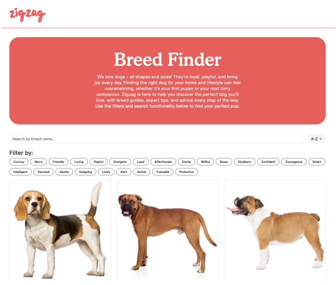

Candidate: James Pope

Date: 6th October 2025

Front End

To improve usability and ‘visual consistency’ with ZigZag’s branding, I made a few updates:

- Moved the ZigZag logo to the left to match the company website.
- Borrowed ZigZag’s fonts (Cooper and Helvetica Now) for a consistent look.
- Added a meta description, page title, and favicon for SEO, readability, and brand recognition.

Hero Box

- Added a Hero box to inform visitors this is a “Breed Finder.” - I’ve kind of assumed this is to find a puppy.
- The box highlights ZigZag’s love for dogs and reassures users about the purpose of the page.
- Provides a clear call-to-action to use the filters and search functionality below *(in bold)*.

Feature 0: Breed Listing

- Replaced the original simple blue button-only layout with a richer, more informative grid.
- Each breed card shows an image, name, description, size, origin, life expectancy, and temperament.

Feature 1: Search Box

- Allows searching for a breed by name.
- Displays an error message when no breeds match.
- Includes placeholder text
- Added an A-Z sorting toggle with icon

Feature 2: Temperament Pills

- Allows filtering breeds by personality traits.
- Supports multiple selections, displaying breeds that match any selected traits.

Technical Improvements / Code Organisation

- Moved breed.service.ts to a services folder for maintainability and scalability (future proofing when adding more services).
- Introduced a dedicated component folder and extracted an example component (HeroBox) for single-responsibility, easier testing, and future reuse.
- Lazy-loaded breed images for performance as more images of breeds are added.
- Defined Breed and BreedState interfaces for typed, predictable state management.

- A reducer function centralises state updates, combining search, temperament filtering, and sorting so users can interact with multiple filters simultaneously.
- Ran tests across the monorepo - all pass, but further work is needed to make them production-ready, including writing more meaningful tests (given the 3–4 hour timeframe).

Backend

- Exported BreedService from BreedModule to allow reuse in other modules, improving scalability and maintainability.
- Introduced a repository abstraction to decouple the service from the data source. Still using JSON, but this design allows easy swap to a PostgreSQL/TypeORM repository without changing service or controller.
- Reorganised backend folders into controllers, services, repositories and models to improve folder structure for maintainability, and scalability.

Next Steps:

Visual / Design

- Refine the look and feel to fully align with ZigZag's design system / figma.
- Ensure consistent spacing, typography, and colors for a polished production-ready interface.
- Consider perhaps a side-by-side breed comparison page (similar to Apple's 'Compare iPhone models' page)
- Create a grid/list view toggle button

Features / Functionality

- Add user personalisation (save favourites, recommended breeds).
- Possibly extend API with richer data points: popularity, ease of training, responsiveness, more images?

Testing / Quality

- Review tests, add more meaningful tests and spec files for reducers and components to verify actions and selectors.
- Run SonarCloud, linting, and A/B tests for UX and Hero messaging.
- Track filter/search usage, page load speed, SEO, and user interactions etc

Architecture / Scalability

- Introduce DTOs and implement a PostgreSQL-backed repository using TypeORM.
- Improve the Nx/monorepo structure by moving reusable Angular components into a shared-ui library and shared models into a shared-data library for easier maintenance and reuse across apps.